

Week 5 Assignment

Name : Payal Vijay Rananaware

Note : In this week , We don't have any specific dataset mention for use, so I use Superset dataset which I used for 2nd week assignment. So in some question query is changed depend on column names.

Q1: What are the key limitations of traditional MapReduce that make **Spark** a preferred choice for modern big data processing?

Traditional MapReduce has several limitations that make it less suitable for modern data processing needs. It is primarily disk-based, which means that after every operation, intermediate results are written to disk.

This increases processing time significantly. It is also not efficient for iterative tasks such as machine learning and data analytics because each iteration requires repeated disk access. Additionally, MapReduce requires complex and lengthy coding for even simple operations, making development slower.

Due to these limitations, Apache Spark is preferred because it performs in-memory computation, reduces execution time, and supports advanced analytics and real-time processing.

Q2: Explain how Spark uses **In-Memory Computing** to speed up iterative machine learning algorithms compared to disk-based systems.

Apache Spark uses in-memory computing to improve performance by storing intermediate data in RAM instead of writing it to disk.

This allows faster data access and reduces the time required for repeated operations. In iterative machine learning algorithms, the same dataset is processed multiple times.

Spark keeps this data in memory, which avoids repeated disk input and output operations.

As a result, computations are significantly faster compared to traditional disk-based systems like MapReduce.

Q3: Write a code snippet to remove all **duplicate rows** from a DataFrame based on a specific set of columns: `user_id` and `transaction_date`.

```
[263] df_clean = df_clean.dropDuplicates(["Customer ID", "Order Date"])
✓ 0s
```

```
[264] df_clean.show(5)
✓ 0s
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1300	CA-2015-121391	10/4/2015	10/7/2015	First Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94109	West	OFF-ST-10001590
5199	CA-2016-103982	3/3/2016	3/8/2016	Standard Class	AA-10315	Alex Avila	Consumer	United States	Round Rock	Texas	78664	Central	OFF-SU-10000151
2230	CA-2014-128055	3/31/2014	4/5/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94122	West	OFF-BI-10004390
1160	CA-2017-147039	6/29/2017	7/4/2017	Standard Class	AA-10315	Alex Avila	Consumer	United States	Minneapolis	Minnesota	55407	Central	OFF-AP-10000576
7469	CA-2014-138100	9/15/2014	9/20/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	New York City	New York	10011	East	OFF-PA-10000349

only showing top 5 rows

```
df_clean = df_clean.dropDuplicates(["Customer ID", "Order Date"])
df_clean.show(5)
```

me	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category	Product Name	Sales	Quantity	Discount	Profit
la	Consumer	United States	San Francisco	California	94109	West	OFF-ST-10001590	Office Supplies	Storage	Tenex Personal Pr...	26.96	2	0	7.0096
la	Consumer	United States	Round Rock	Texas	78664	Central	OFF-SU-10000151	Office Supplies	Supplies	High Speed Automa...	3930.072	3	0.2	-786.0144
la	Consumer	United States	San Francisco	California	94122	West	OFF-BI-10004390	Office Supplies	Binders	GBC DocuBind 200 ...	673.568	2	0.2	252.588
la	Consumer	United States	Minneapolis	Minnesota	55407	Central	OFF-AP-10000576	Office Supplies	Appliances	Belkin 325VA UPS ...	362.94	3	0	90.735
la	Consumer	United States	New York City	New York	10011	East	OFF-PA-10000349	Office Supplies	Paper	Easy-staple paper	14.94	3	0	7.0218

Q4: Given a DataFrame `df_sales`, write a query to **filter** for rows where the region is 'West' and then **group by** `product_category` to find the average `sale_amount`.

```
from pyspark.sql.functions import col, avg

df_clean.filter(col("Region") == "West") \
    .groupBy("Category") \
    .agg(avg("Sales").alias("Average_Sales")) \
    .show()
```

```
... +-----+
|      Category|      Average_Sales|
+-----+-----+
|Office Supplies|102.71960021786482|
|Furniture|344.26522927461116|
|Technology|389.98297769516756|
+-----+-----+
```

Q5: What is the difference between `.na.drop()` and `.na.fill()`? Provide a code example of filling null values in a status column with the string 'Unknown'.

The `.na.drop()` function is used to remove rows from the dataset that contain null or missing values. When this method is applied, any row with at least one null value is deleted from the DataFrame.

This helps in ensuring that only complete and valid records are used for further analysis, but it may lead to loss of data if many rows contain missing values.

On the other hand, the `.na.fill()` function is used to replace null or missing values with a specified value instead of removing the entire row.

This method is useful when we want to preserve the dataset while handling missing values by assigning default values such as zero, mean, or a specific string.

In summary, `.na.drop()` removes incomplete records, whereas `.na.fill()` retains the records by replacing missing values with appropriate substitutes.

```
df_clean = df_clean.na.fill({
    "Segment": "Unknown"
})

df_clean.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1300	CA-2015-121391	10/4/2015	10/7/2015	First Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94109	West	OFF-ST-10001590
5199	CA-2016-103982	3/3/2016	3/8/2016	Standard Class	AA-10315	Alex Avila	Consumer	United States	Round Rock	Texas	78664	Central	OFF-SU-10000151
2230	CA-2014-128055	3/31/2014	4/5/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	San Francisco	California	94122	West	OFF-BI-10004390
1160	CA-2017-147039	6/29/2017	7/4/2017	Standard Class	AA-10315	Alex Avila	Consumer	United States	Minneapolis	Minnesota	55407	Central	OFF-AP-10000576
7469	CA-2014-138100	9/15/2014	9/20/2014	Standard Class	AA-10315	Alex Avila	Consumer	United States	New York City	New York	10011	East	OFF-PA-10000349

only showing top 5 rows

Q6: Write a query to find the **total count** of records for each city in a DataFrame, but only for cities where the count is greater than 100.

3.2 Count records per City > 100

```
df_west.groupBy("city") \
        .count() \
        .filter("count > 100") \
        .show()
```

```
... +-----+-----+
    |           City|count|
    +-----+-----+
    | Los Angeles|   731|
    |San Francisco|   494|
    |   San Diego|   166|
    |     Seattle|   412|
    +-----+-----+
```

Q7: How does the **immutability** of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are immutable, which means they cannot be changed after creation. Any operation such as dropping or renaming a column does not modify the original DataFrame. Instead, a new DataFrame is created with the applied changes. This ensures data consistency and avoids unintended modifications during processing.

Q8: Write a Spark command to **filter** a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

(Change in query, depending upon column present in Dataset)

```
from pyspark.sql.functions import col

df.filter(col("Segment") == "Consumer").show()
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-1
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-1
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-1
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-1
6	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	FUR-FU-1
7	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	OFF-AR-1
8	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-1
9	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	OFF-BI-1
10	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	OFF-AP-1
11	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	FUR-TA-1
12	CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	Brosina Hoffman	Consumer	United States	Los Angeles	California	90032	West	TEC-PH-1
13	CA-2017-114412	4/15/2017	4/20/2017	Standard Class	AA-10480	Andrew Allen	Consumer	United States	Concord	North Carolina	28027	South	OFF-PA-1
14	CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	Irene Maddox	Consumer	United States	Seattle	Washington	98103	West	OFF-BI-1
17	CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	Pete Kriz	Consumer	United States	Madison	Wisconsin	53711	Central	OFF-ST-1
18	CA-2014-167164	5/13/2014	5/15/2014	Second Class	AG-10270	Alejandro Grove	Consumer	United States	West Jordan	Utah	84084	West	OFF-ST-1
19	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	California	94109	West	OFF-AR-1
20	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	California	94109	West	TEC-PH-1
21	CA-2014-143336	8/27/2014	9/1/2014	Second Class	ZD-21925	Zuschuss Donatelli	Consumer	United States	San Francisco	California	94109	West	OFF-BI-1
24	US-2017-156909	7/16/2017	7/18/2017	Second Class	SF-20065	Sandra Flanagan	Consumer	United States	Philadelphia	Pennsylvania	19140	East	FUR-CH-1
25	CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	Emily Burns	Consumer	United States	Orem	Utah	84057	West	FUR-TA-1

only showing top 20 rows

Q9: When **cleaning a dataset**, why is it often better to handle null values before performing mathematical aggregations like `sum()` or `avg()`?

Handling null values before performing aggregations is important because null values can affect the accuracy of results.

Aggregation functions may ignore nulls or return incorrect values depending on the operation. Cleaning the data beforehand ensures that the computed results such as sum and average are reliable and meaningful.

Q10: Write the code to **revise** a column named `raw_timestamp` by casting it to a `TimestampType` and renaming it to `event_time`.

```
from pyspark.sql.functions import to_timestamp, col

df = df.withColumn(
    "event_time",
    to_timestamp(col("Order Date"), "M/d/yyyy")
)

df.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

only showing top 5 rows

Q11: Explain the "Shuffle" process that occurs during a **grouping** operation. Why is it considered a wide transformation?

Shuffle is the process of redistributing data across different partitions in a cluster. It occurs during operations such as `groupBy` and `join`.

During this process, data is moved between nodes to bring related records together. It is considered a wide transformation because it involves data movement across partitions rather than operating within a single partition, which increases execution cost.

Q12: Write a code snippet that identifies and **removes rows** where the email column contains null values OR the username is an empty string.

```
from pyspark.sql.functions import col

df_clean = df.filter(
    (col("Customer Name").isNotNull()) & (col("Customer Name") != "")
)

df_clean.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

only showing top 5 rows

Q13: How do you use the `.agg()` function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

The `agg` function allows multiple aggregations in a single operation. It can compute different statistical measures at once.

Step 3.4 – Multiple Aggregations

```
from pyspark.sql.functions import sum, avg, max, min

df_west.groupBy("Category") \
    .agg(
        sum("Sales").alias("Total_Sales"),
        avg("Sales").alias("Avg_Sales"),
        max("Sales").alias("Max_Sales"),
        min("Sales").alias("Min_Sales")
    ) \
    .show()
```

Category	Total_Sales	Avg_Sales	Max_Sales	Min_Sales
Office Supplies	213125.18300000002	117.48907552370453	8187.65	1.08
Furniture	248450.23350000026	360.59540420899896	3610.848	3.48
Technology	251895.92799999993	422.64417449664415	13999.96	0.99

Q14: In the context of **cleaning a dataset**, what is the risk of using `inferSchema=true` when your source data contains messy or inconsistent date formats?

Using `inferSchema` automatically detects data types, but it may incorrectly interpret inconsistent or messy data formats.

For example, date columns with different formats may be misinterpreted as strings or null values.

This can lead to incorrect analysis and data quality issues. Therefore, defining the schema manually is often safer.

Q15: Write a final processing pipeline that:

1. Filters out **duplicates**.

Step 2: Data Cleaning

```
df_clean = df.dropDuplicates()

from pyspark.sql.functions import col, when, trim

df_clean = df.withColumn(
    "Sales",
    when(trim(col("Sales")) == "", None)
    .when(trim(col("Sales")).rlike("[0-9.]+$"), col("Sales"))
    .otherwise(None)
    .cast("double")
).withColumn(
    "Quantity",
    when(trim(col("Quantity")) == "", None)
    .when(trim(col("Quantity")).rlike("[0-9]+$"), col("Quantity"))
    .otherwise(None)
    .cast("int")
)

df_clean = df_clean.dropna()
```

2. Fills null prices with 0.

```
df.show(5)
```

Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID	Category	Sub-Category
'11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798	Furniture	Bookcases
'11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454	Furniture	Chairs
'16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240	Office Supplies	Labels
'18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577	Furniture	Tables
'18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760	Office Supplies	Storage

```
df_clean = df_clean.na.fill({
    "Sales": 0,
    "Profit": 0
})
```

3. Groups by store_id to calculate total revenue.

```
df.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

only showing top 5 rows

```
df_clean = df_clean.na.drop(subset=["Customer ID", "Order ID"])

df_clean.show(5)
```

Row ID	Order ID	Order Date	Ship Date	Ship Mode	Customer ID	Customer Name	Segment	Country	City	State	Postal Code	Region	Product ID
1	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-BO-10001798
2	CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	Claire Gute	Consumer	United States	Henderson	Kentucky	42420	South	FUR-CH-10000454
3	CA-2016-138688	6/12/2016	6/16/2016	Second Class	DV-13045	Darrin Van Huff	Corporate	United States	Los Angeles	California	90036	West	OFF-LA-10000240
4	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	FUR-TA-10000577
5	US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	Sean O'Donnell	Consumer	United States	Fort Lauderdale	Florida	33311	South	OFF-ST-10000760

only showing top 5 rows

